

Spike: Collecting Payment on Intake-Form Submission (M-Pesa, Pesapal)

Status: spike / research only — no code changes. Prepared: 2026-07-30

1. Current state

Nothing charges anyone today. The flow is: landing page (Vercel) → "Start My Plan" CTA → loading page → `career-transition-intake.onrender.com` Flask app → user fills the intake form → `POST /submit` validates input, builds a PDF, emails it to the consultant via Resend, and streams the PDF back to the browser. The landing page now advertises an introductory price — KES 7,500 (50% off KES 15,000) — stated as "paid when you submit your intake form," but nothing in `app.py` collects or checks for payment yet.

Goal of this spike: figure out how to actually collect that fee at the moment of submission, starting with M-Pesa and Pesapal, without committing to code yet.

2. Provider options

2.1 M-Pesa Daraja API (direct, Safaricom)

Requires the consultant to personally hold a Till (Buy Goods) or Paybill shortcode — there's no path to STK Push without one. A Till, applied for as an individual/sole-proprietor via Safaricom's self-onboarding portal, can go live in 24–72 hours; a Paybill needs a signed application form dropped at a Safaricom shop and takes 5–10 working days, with heavier KYC/business-use-case review. Either way, going from shortcode → sandbox build → production go-live approval is a separate step (register the app on `developer.safaricom.co.ke`, get sandbox credentials, build, then apply for production access with a valid HTTPS callback URL). Realistic end-to-end timeline for a first-timer: 4–8 weeks, not days. Safaricom is also mid-rollout of Daraja 3.0 (announced Nov 2025), so docs/endpoints are in some flux right now.

The API flow itself (STK Push): OAuth token → `POST /mpesa/stkpush/v1/processrequest` → Safaricom responds synchronously with a `CheckoutRequestID` (not the result) → the actual result arrives later as an async POST to your callback URL, which must be a publicly reachable HTTPS endpoint — `onrender.com`'s default HTTPS is sufficient, no custom domain needed just for this. Once production credentials exist, the code itself is maybe 2–4 days of work for a solo Flask dev (OAuth handling, STK push initiation, callback route, `STKPushQuery` polling fallback, phone-number normalization, handling the documented STK result codes).

Fees are the standard M-Pesa merchant tariff — cheapest of all options (roughly <1%, tiered by shortcode type, negotiable at volume) — but the bureaucratic runway makes this a poor starting point.

2.2 Pesapal

Pesapal aggregates M-Pesa, Airtel Money, and cards under one API — it already holds its own Daraja relationship and lets you onboard as a sub-merchant, so it skips the shortcode application entirely. Sole-proprietor onboarding needs ID/passport, KRA PIN + tax compliance certificate, business registration if any, and a bank confirmation/voided cheque — lighter than getting your own Paybill/Till, though still a human-reviewed KYC step (no confirmed public SLA; treat as days to ~2 weeks).

Integration is API v3, JSON/REST, IPN-based: register a notification URL once (RegisterIPNURL), then per order call SubmitOrderRequest, which returns a redirect URL to Pesapal's hosted checkout page where the customer picks M-Pesa/card/Airtel. Important gotcha: neither the browser redirect back nor the IPN payload carries the actual payment status "for security reasons" — you must call GetTransactionStatus yourself using the OrderTrackingId to learn success/failure. This is materially simpler than direct Daraja (no OAuth lifecycle, no STK phone-number handling, one API covers three payment methods) — roughly 1–2 days of Flask work.

Fees run ~1.5–3.5% depending on method (M-Pesa at the low end, cards higher) — more expensive than direct Daraja because Pesapal's margin sits on top of the underlying M-Pesa fee, but that's the trade for shipping now instead of in 4–8 weeks.

2.3 Other aggregators worth a same-day look

- IntaSend — Nairobi-based, explicitly markets itself as avoiding "the

long Daraja API approval process," with indications it can accommodate unregistered businesses/freelancers. Exact fee % wasn't confirmed in research (marketed as "no monthly fees, pay per transaction") — worth a direct pricing-page check since it could undercut Pesapal and be even more solo-consultant-friendly.

- Flutterwave — M-Pesa support without a personal Daraja integration;

needs certificate of registration (or e-Citizen business name cert), ID, KRA PIN. KYC turnaround reported 1–3 business days. Fees ~1.4–1.5% M-Pesa.

- Paystack — now live for all Kenyan merchants (2025/2026 expansion),

KES settlement, M-Pesa + card + Apple Pay. Fees: 1.5% M-Pesa, 2.9% local card.

Rough fee ranking for M-Pesa collection: direct Daraja (<1%, weeks of setup) < Flutterwave/Paystack (~1.4–1.5%) < Pesapal (1.5–3.5%).

3. Recommended integration pattern

Regardless of provider, /submit needs to become a two-step flow instead of one:

1. Validate the form fields/files as today, but **hold** them server-side in a

pending state (keyed by a generated order ID) instead of immediately building the PDF/emailing/streaming a download.

2. Trigger payment before running any business logic — for Pesapal, call

SubmitOrderRequest and redirect the browser to its hosted checkout (or embed as an iframe); for direct M-Pesa, fire an STK Push and show a "check your phone" waiting state.

3. Treat payment success as confirmed only via the server-to-server

channel (Daraja callback, or Pesapal IPN + GetTransactionStatus) — never trust the browser redirect alone, since the browser can be closed or tampered with.

4. Only build the PDF / email it / allow download once the webhook/callback

marks that order as paid, and do so idempotently (a duplicate callback delivery must not double-email or double-process).

Failure modes worth designing for up front: the user closing the tab mid-payment (harmless if state is server-side and keyed by order ID — uploaded files need to persist in temp/object storage long enough to survive the gap); STK Push timeouts (~60-100s window, needs a query-based fallback and a retry path); and — specific to this app's current hosting — a webhook/callback landing on a cold Render free-tier instance. Render spins down after ~15 minutes idle and takes ~30-60s to wake, which risks a Safaricom/Pesapal callback timing out or being dropped depending on the provider's retry policy. Once payment is live, this argues for either upgrading the Render service off the free tier or leaning on query-based status checks (STKPushQuery / GetTransactionStatus) as the source of truth rather than depending solely on the push callback arriving.

4. Logging transactions to a Google Sheet

The consultant wants every payment attempt — success or failure — recorded as a row in a Google Sheet, as a simple human-readable ledger they can open and eyeball without a database or admin dashboard. This is genuinely new integration work, not an extension of the existing `extras/onboard_metrics.xlsx` script: that script writes a local `.xlsx` file on the dev's own machine, whereas Render's filesystem is ephemeral and has no persistent disk, so the Flask process itself can't write a durable spreadsheet file — it needs to call out to a live Google Sheet over the network instead.

Recommended approach: Google Sheets API v4 via the `gspread` library, using a service account, with the JSON key stored as a Render Secret File.

Setup is one-time and code-free: create a GCP project (no billing required — Sheets API is free within standard quotas), enable the Sheets API, create a service account and download its JSON key, then share the target Sheet with the service account's email as an Editor — exactly like sharing with a human Google account. Quotas (300 write requests/min per project, 60/min per user) are three orders of magnitude beyond what a solo consultant's transaction volume will ever approach, so this is a non-issue at this scale.

`gspread` reduces the actual code to roughly 15-30 lines: load the service- account JSON, open the sheet by ID, and call `append_row([timestamp, client_name, amount, provider, txn_id, status])`. This is the same shape of integration the app already has for Resend — call an external API using a credential pulled from Render's secret storage — so it extends a pattern the codebase already understands rather than introducing a new one.

For the credential itself, Render's Secret Files feature (dashboard → service → Environment → "+ Add Secret File") is a better fit than the usual base64-encode-into-an-env-var workaround other platforms force: the JSON key is mounted as a real file at runtime (`/etc/secrets/...`), so `gspread.service_account(filename=...)` just works, with no encode/decode step and no risk of the credential blob leaking into a log dump of `os.environ`.

An alternative — a Google Apps Script doPost Web App that Flask calls via `plain requests.post()` — avoids GCP/service-account setup entirely, but shifts security into a hand-rolled shared-secret check inside the script (the deployed URL is otherwise publicly POST-able) and adds a second deploy/versioning lifecycle to maintain. For something logging financial transactions, the service-account model's proper IAM-based access control is the better fit; not recommended over the `gspread` approach.

Failure handling: the Sheets append is a side-effect log, not the system of record — the payment provider's own webhook/dashboard remains the authoritative record of whether a payment succeeded. The append call should never block or delay marking an order paid or sending the PDF via Resend: wrap it in a short-timeout try/except, log-and-move-on on failure, no retry queue needed at this volume (Redis/Celery would be over-engineering for a solo consultant's transaction count). An occasional dropped ledger row is a minor inconvenience, not a financial-integrity problem.

Added effort on top of the M-Pesa/Pesapal work: one-time setup (GCP project, service account, share sheet, add Render Secret File) is roughly 20–30 minutes with no code; the code itself is one small helper module plus 1–2 call sites in the payment webhook handler — well under an hour total, and low-risk given the fire-and-forget failure handling above.

5. Recommendation

Start with an aggregator, not raw Daraja. The consultant is a solo/ informal operator; the Paybill/Till + go-live path is realistically weeks of bureaucracy before a single payment can be collected, which blocks revenue the whole time it's pending. Pesapal is the safest first pick — Kenya- first, confirmed to cover M-Pesa + Airtel + cards in one API, a documented sole-proprietor KYC path, and genuinely simple API v3/IPN integration (~1–2 days of Flask work). Its higher M-Pesa fee (up to 3.5%) is a fair trade for shipping now.

Before committing, spend a same-day check on IntaSend's actual sign-up flow and fee sheet — it may be friendlier still to an unregistered solo business and could replace Pesapal as the default if its rate is meaningfully lower.

Treat direct Daraja integration as a later optimization once the business is formally registered and volume justifies chasing the lower fee — not a starting point.

For the ledger, use gspread with a service account and a Render Secret File, not the Apps Script webhook alternative — it reuses the app's existing "external API + Render-stored credential" pattern and gives proper IAM-based access control on something logging financial transactions.

6. Open questions before building

- Is the consultant operating as a registered sole proprietorship, or fully

informal? This determines which KYC documents are actually available and therefore which providers are realistically approvable in the near term.

- Confirm IntaSend's real fee % and onboarding flow directly — not well

documented publicly.

- Decide hosted-checkout redirect vs. embedded iframe for Pesapal (redirect

is less code; iframe keeps the user on-domain, which matters more once the platform-unification spike below is acted on).

- Decide whether to fix Render's free-tier cold-start behavior (upgrade plan,

or keep-alive ping) before or alongside going live with payments, since a missed webhook now means a missed payment confirmation, not just a slow page load.

Sources

- [Safaricom Daraja Developer Portal](#)
- [Lipa na M-Pesa Requirements 2024 PDF](#)
- [How to Go Live on M-Pesa Daraja API](#)
- [Pesapal Developer Community — RegisterIPNURL](#)
- [Pesapal Developer Community — SubmitOrderRequest](#)
- [Pesapal — documents required for a Pesapal account](#)

- [Pesapal — merchant registration procedure](#)
- [Safaricom overhauls M-Pesa API platform with Daraja 3.0 — TechCabal](#)
- [IntaSend M-Pesa API](#)
- [Flutterwave onboarding requirements — Kenya](#)
- [Paystack is live for all merchants in Kenya](#)
- [Paystack pricing — Kenya](#)
- [Render Docs — Web Services](#)
- [Render Docs — Free Tier](#)
- [Google Sheets API — `spreadsheets.values.append`](#)
- [Google Sheets API — Usage limits](#)
- [Google — Create access credentials](#)
- [gsread — Authentication](#)
- [gsread on PyPI](#)
- [Render Docs — Environment Variables and Secrets \(Secret Files\)](#)
- [Render API Docs — update secret file](#)
- [Google Apps Script — Web Apps](#)
- [Google Apps Script — Quotas for Google Services](#)